WEEK 4 · DISCOVER PHASE

Design with Agents

Turn last week's backlog into **design** — feature-list, user-journey, prototype, architecture and the **4 required diagrams** — using AI agents, then review like an engineer.

Deliverable: design (feeds W5 Gate)

Case: **CupLog**

Dr. Prasara Jakkaew · ADT MFU

RECAP · WHERE WE ARE

From backlog to design

W1 · Charter → W2 · Legal / `rule.md` → W3 · Requirements → Backlog → **W4 · Design** → W5 · Gate

YOU ALREADY HAVE (FROM W3)

- Requirement spec + **product backlog** (INVEST user stories)
- `CLAUDE.md` · `rule.md` · agents & skills on GitHub
- Interview evidence (users ≥ 5, still growing)

TODAY YOU PRODUCE

- feature-list · user-journey · prototype
- architecture + **4 diagrams** (D1–D4)
- a small **design system** the agent builds the prototype from

This is **Stage 2 (Design → spec.md)** of the AI-Native SDLC playbook — you turn intent + backlog into a reviewed design before any build.

THE DESIGN DELIVERABLE

Five artifacts, one folder

.docs/02-design/

```
02-design/
  feature-list.md      # features, core one marked
  user-journey.md      # the 1 core workflow, step by step
  design-system.md     # tokens + component rules (NEW today)
  prototype.md         # lo-fi screens (agent builds from the system)
  diagrams.md          # D1 Context · D2 Use Case · D3 Arch · D4 Activity
```

💡 **IN PLAIN TERMS** Each file answers **one question**: feature-list = *what can it do?* · user-journey = *how does one task flow?* · design-system = *what does it look like?* · prototype = *show me the screens* · diagrams = *prove the parts fit together*.

Rule of the day: every one of these describes the **same one core workflow**. If your diagrams, journey and prototype disagree, you lose marks — consistency is the whole game.

THE ONE THING THAT EARNS MARKS

One core workflow — everything agrees

Pick **one** core workflow (for CupLog: `"Record a cupping"`). Every artifact must trace back to it and to a requirement.

pain (interview) → user story **F1** → feature "Record a cupping" → journey 5 steps → D1–D4 diagrams → prototype screen

EARNS MARKS

- Same actor names everywhere
- Diagram flow = user-journey = requirement
- Architecture matches the tech stack you named

LOSES MARKS

- Diagram copied from a blank template
- Actors / labels not in your spec
- Journey says one thing, D4 another

💡 **IN PLAIN TERMS** Pick **one task** and follow it like a single thread through every document. If the thread breaks — a diagram shows a step your journey doesn't — you lose marks. **Example:** pain "I lose my cupping notes" → story F1 → feature "Record a cupping" → 5-step journey → D4 → the New-cupping screen. One thread, six documents.

STEP 1 · WITH AN AGENT

Backlog → feature-list

prompt to your design agent

```
Read backlog.md and the requirement spec.
Group the user stories into a short
feature-list for the DISCOVER design.
Mark exactly ONE core feature (the one
we build first). Keep it to 5–7 lines.
Output: 02-design/feature-list.md
```

feature-list.md (CupLog)

```
- Record a cupping (score + tags)  ← core
- Add a coffee
- Shared flavor-tag picker
- History list (view, filter, sort)
- Consent + login (with access log)
```

YOU REVIEW, NOT JUST ACCEPT

- Is the **core** feature really the one that kills the top pain?
- Does every feature trace to a story in `backlog.md`?
- Cut anything not needed for month-2 build (scope!)

The agent drafts; **you own the judgment**. Same pattern all day.

💡 IN PLAIN TERMS A feature-list is your backlog boiled down to "**what the product can DO**", with a star ⭐ on the first thing you'll build. The agent writes it in seconds — your job is to check the star is on the right feature.
Example: CupLog's core = "Record a cupping" because that's the pain users named most.

STEP 2 · WITH AN AGENT

The core user-journey

user-journey.md (CupLog — "Record a cupping")

1. Barista opens CupLog on the phone
2. Picks a coffee from the list
3. Gives a score (1–10) + taps 3–5 flavor tags
4. Saves → sees a confirmation
5. Opens History → new cupping on top

WHY THE JOURNEY COMES BEFORE DIAGRAMS

- It fixes the **steps** D4 (Activity) must show
- It names the **screens** the prototype needs
- It exposes decisions (online? consent?) early

Prompt: "From feature-list + spec, write the ONE core workflow as 5 numbered steps, mobile-first."

💡 IN PLAIN TERMS A user-journey is the **numbered story of one task**, told from the user's side — no tech words, just what they tap and see. **Example:** the 5 steps a barista does to record one cupping, phone in hand.

Keep it to the **happy path** plus the one decision that matters. Edge cases live in the backlog, not the journey.

STEP 3 · THE REQUIRED SET

The 4 diagrams (D1–D4)

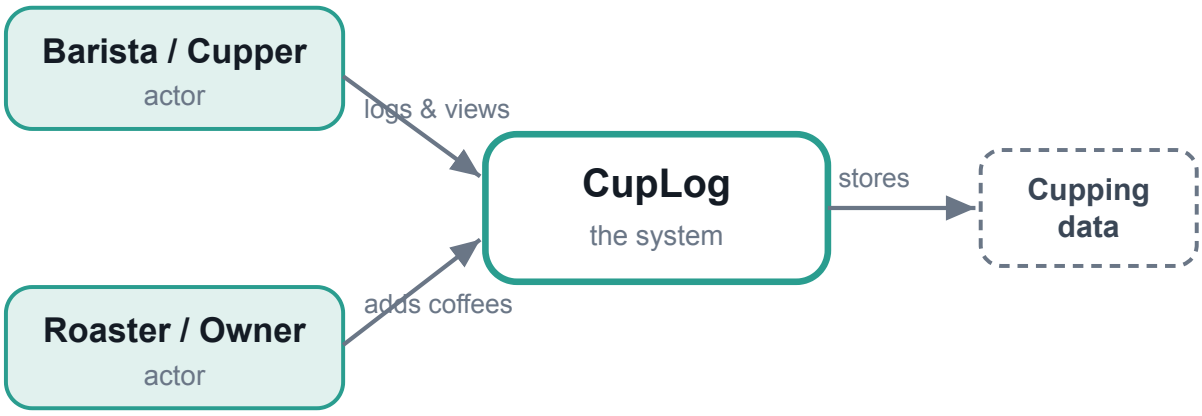
#	DIAGRAM	MUST SHOW	CORRECT IF…
D1	**System Context**	system · external actors · scope	named system · actors ≥ 2 · no internal detail
D2	**Use Case** / User Story Map	main use cases · actors · include/extend	actors linked to use cases · covers core workflow
D3	**Architecture** (high-level)	layers/components · data or call direction	doesn't contradict your tech stack (§5)
D4	**Activity** / Sequence	1 scenario from the core user story	start/end · matches requirement · clear order

💡 IN PLAIN TERMS Four pictures, four zoom levels of the **same product**: **D1** the system from far away · **D2** who uses it & for what · **D3** the parts inside · **D4** the steps of one task. Same coffee shop, four camera angles.

Tools: draw.io · Lucidchart · Figma · PlantUML — attach the source file in the appendix.
Forbidden: blank-template copies · labels not in your docs · missing legend when needed.

D1 · SYSTEM CONTEXT

The system as one box



WHAT EARNS THE MARK

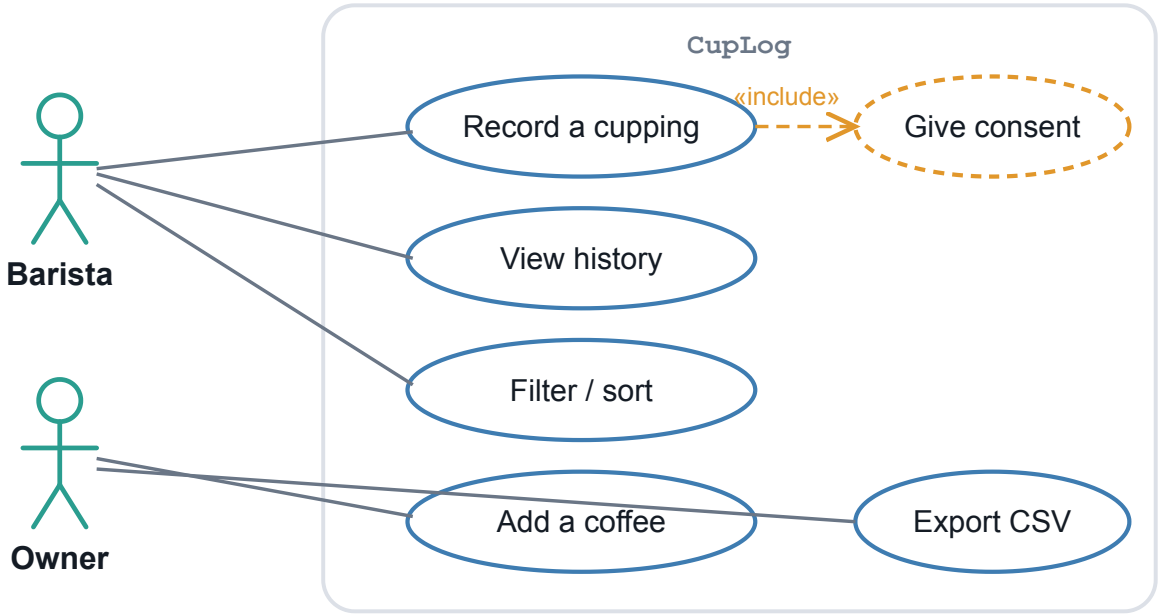
- The system is **one box** — no inside detail
- ≥ 2 external actors, named as in your spec
- Scope is obvious: what's in, what's out

💡 READ IT Draw the app as **one box**, plus everyone/everything outside that talks to it. No insides yet — just the boundary. **Here:** the CupLog box, the Barista & Owner who use it, and where the data is stored.

Agent prompt: "`Draw D1 System Context for CupLog from the spec — system + external actors + stored data only. Output mermaid.`"

D2 · USE CASE / USER STORY MAP

Who can do what



WHAT EARNS THE MARK

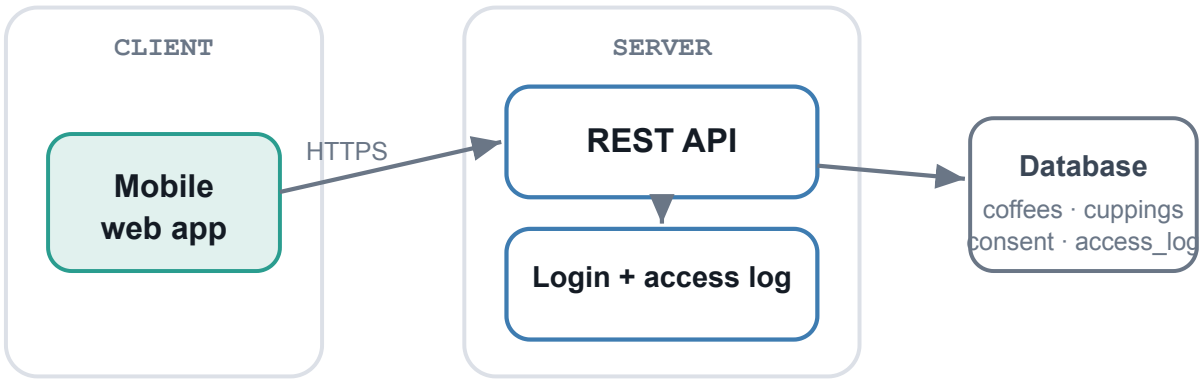
- Every actor is **linked** to its use cases
- The **core** use case is present & central
- include/extend used only when real (record «includes» consent)

💡 READ IT **Stick people** (actors) linked to **bubbles** (things they can do). **Here:** the Barista can "Record a cupping"; recording always *includes* "Give consent".

UML notation: actors = **stick figures** · use cases = ovals inside the system box · actor–use-case = **plain solid line (no arrowhead)** · «include»/«extend» = **dashed line, open arrowhead**. A User Story Map is an accepted alternative — pick what fits your product (chatbot vs. CRUD vs. ranking).

D3 · HIGH-LEVEL ARCHITECTURE

Boxes, layers, and the data direction



WHAT EARNS THE MARK

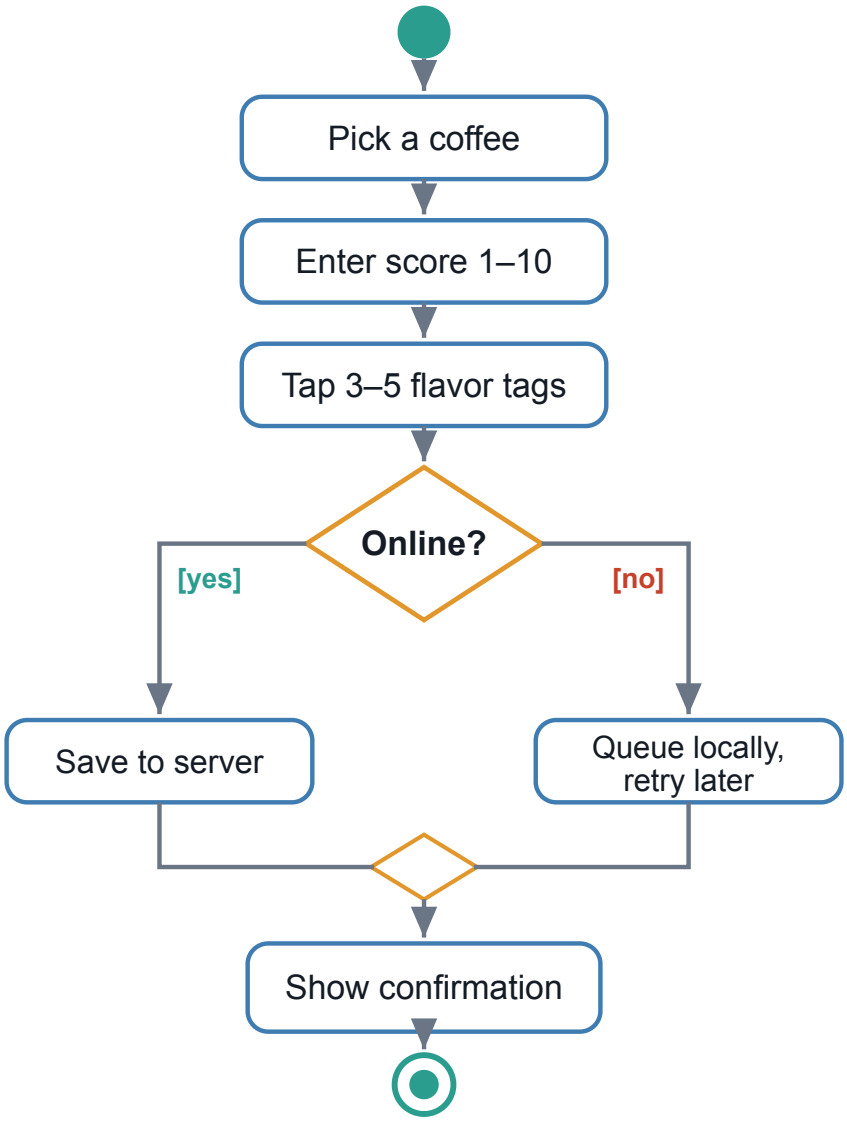
- Layers/components, not code — high level
- Arrows show **data / call direction**
- **Agrees with your tech stack** named in §5

💡 READ IT The **parts inside** and which way data moves: phone → server → database. Boxes are parts; **arrows mean "who calls whom"**. **Here:** the web app calls a REST API, which reads/writes the database.

The `rule.md` duties surface here: an **access_log** table (CCA §26) and a **consent** store (PDPA) belong in this picture.

D4 · ACTIVITY / SEQUENCE

One scenario, start to end



💡 **Read it:** one task as a flowchart with real UML dots — start ● → steps → a diamond ◆ where it decides → end ◉. Here: saving a score, with an "Online?" branch.

WHAT EARNS THE MARK

- **UML nodes, not boxes:** ● initial → actions → ◉ final (never a labelled "Start"/"End" box)
- Decision & merge = **diamond**; branches rejoin at a merge before the final
- Steps = the **user-journey** (same order); guards in `[brackets]`

A **sequence** diagram (lifelines · activation bars · solid call / dashed return) is an accepted alternative for message-heavy flows (e.g. a chatbot).

BEFORE YOU MOVE ON

Diagram self-check — run it as a subagent

`.claude/agents/diagram-checker.md`

```
---
name: diagram-checker
description: Checks D1–D4 agree with the spec
tools: Read
---
Read spec, backlog, user-journey and
diagrams.md. Report ONLY mismatches:
- actors/labels not in the spec
- a journey step missing from D4
- architecture that contradicts the
  tech stack in §5
Do not fix. List findings with file+line.
```

WHY A CHECKER, NOT EYEBALLING

- Consistency is exactly what the Gate grades
- The agent finds the mismatch **before the grader does**
- Same "verify your own work" idea as Stage 4 of the playbook

💡 **IN PLAIN TERMS** A **subagent** is just a mini-helper you define in a small file — here, one whose only job is to compare your diagrams to your spec and **shout the differences**. Like a proofreader that only checks facts and never rewrites your work.

This is the DISCOVER-phase version of a feedback loop: the design checks itself, you fix the doc, not the diagram-of-record.

STEP 4 · THE NEW PIECE TODAY

What is a Design System — and why agentic?

A **design system** = your product's reusable visual rules: **tokens** (color, type, spacing, radius) + **component rules** (buttons, chips, cards). Define it once; every screen obeys it.

WITHOUT A SYSTEM

- Every screen invents its own colors/spacing
- The agent guesses → inconsistent prototype
- Hard to review, harder to build

WITH DESIGN-SYSTEM.MD

- The agent reads it and builds **consistent** screens
- It's a **UI Skill** — same idea as `CLAUDE.md`/skills
- Change a token once → whole prototype updates

💡 **IN PLAIN TERMS** A design system is a **style guide the AI can actually follow**: the exact colors, sizes and rules, written down once. **Example:** "primary button = accent color, buttons ≥ 44px" — so every screen the agent draws matches.

Playbook link: `design-system.md` is to UI what `rule.md` is to law — institutional knowledge the agent applies every time it draws a screen.

DESIGN-SYSTEM.MD · HOW YOU GENERATE IT

Don't hand-write it — give the agent a spec + a screen you like

what you hand the agent

spec.md
the requirements

+

a UI screen example
image · mobile or web

```
# attach: spec.md  +  the screen example
Prompt ▸ "create design system, follow this
file for mobile and web app, for this spec"
```

One prompt covers **both** mobile and web. The screen example is the **taste reference**; the spec keeps it grounded — no guessing a palette from nothing.

→ DESIGN-SYSTEM.MD (AGENT-GENERATED)

Tokens lifted from your screen —

☐ coffee-900 ☐ accent ☐ cream

type / space scale · radius · breakpoints (mobile + web)

Component rules as commands: button, chip ≥44px, never hardcode a hex, mobile-first + web breakpoint.

It's a **draft you review** — fix the palette/rules, then every screen obeys it.

STEP 5 · PROTOTYPE FROM THE SYSTEM

Agent builds lo-fi screens — you review

prompt

```
Read design-system.md and user-journey.md.
Generate a lo-fi HTML prototype of the 3
core screens. Use ONLY the tokens and
component rules. Mobile-first, max-width 480.
Output: 02-design/prototype/
```

the 3 screens (CupLog)

```
1. Coffee list      + "Add coffee"
2. New cupping      score slider · flavor chips · Save
3. History          list · filter bar
```

REVIEW LIKE AN ENGINEER

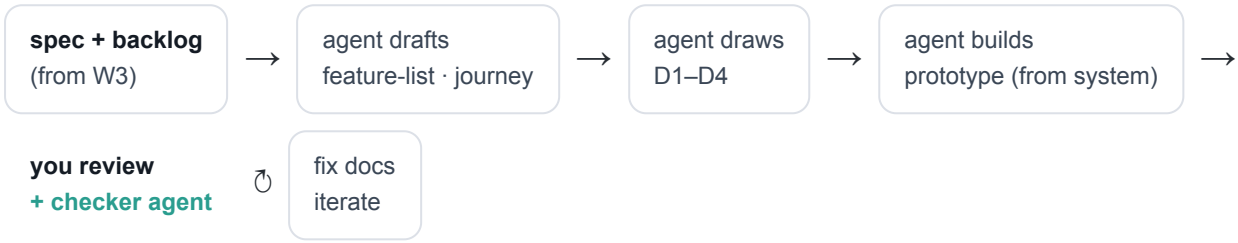
- Do the screens match the **user-journey** steps?
- Did it obey the **tokens** (no rogue colors)?
- Touch targets, max-width, one primary action per screen?

Lo-fi is fine. 2–3 iterations is normal — that's the loop working.

💡 IN PLAIN TERMS A prototype = **rough, clickable screens**, not the final app. The agent builds them from your design-system so they already look consistent. Lo-fi *on purpose* — you're testing the **flow**, not the polish.

PUTTING IT TOGETHER

Design with agents — the mini-loop



Agent does

Drafts artifacts, draws diagrams, generates the prototype from your rules.

You do

Own the core-workflow choice, trace to requirements, judge risk & scope.

Rules do

`rule.md` + `design-system.md` keep every output consistent.

💡 IN PLAIN TERMS The whole day in one sentence: **the agent drafts, you judge, and your rule files keep everything consistent** — then you loop until every document agrees.

Same shape as the AI-Native SDLC playbook: **agent writes, you approve, config keeps it consistent.**

IN-CLASS WORKSHOP · ~45 MIN

Your team's design pack

PRODUCE TODAY (YOUR OWN PRODUCT)

- feature-list.md — core feature marked
- user-journey.md — 1 core workflow, 5 steps
- **design-system.md** — ≥ 5 tokens + ≥ 3 component rules
- D1–D4 diagrams (draw.io / Figma / PlantUML)
- prototype — 3 lo-fi screens from the system

PASS / FAIL (IN-CLASS)

- All 4 diagrams present & **consistent** with the spec
- Diagrams trace to the same core workflow
- Prototype obeys the design system

Photograph / push before you leave = in-class case.

Common traps I'll walk around and catch: diagram copied from a template · actor names not in your spec · design-system that's just colors with no rules the agent can act on.

NEXT · W5 GATE (9 ก.ย.)

Bring a design that agrees with itself

W5 is a **pass/fail Gate**: proposal + backlog + **design (4 diagrams)** + compliance. Missing 1 diagram = CONDITIONAL; inconsistent diagrams = FAIL. Consistency is everything.

Bring: 4 diagrams · prototype · design-system.md

Deliverable folder: **.docs/02-design/**

1305493 · W4 · CupLog worked example on GitHub